Übungsblatt 1: Zeitverträge, Polling und eine Tasteranzeige

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Zeitvertrag einer Verpackungsprüfung (8 Punkte)

Ein Prüfsystem muss nach einer erkannten falschen Etikettierung binnen $D = 7\,\text{ms}$ den richtigen Sperrauftrag am Softwareausgang ausgeben. Kein Miss ist erlaubt; die mechanische Bewegung gehört nicht zum Vertrag. Ein Kontrollfoto ist nach 60 ms wertlos, zwei ausgelassene Fotos je 500 sind zulässig. Eine verspätete Produktionsstatistik bleibt nützlich.

1. Formulieren Sie Anfang, Ende und Zeitbedingung des Sperrauftrags. (2 Punkte)
2. Ordnen Sie die drei Leistungen Hard, Firm und Soft zu und begründen Sie. (3 Punkte)
3. Freigabe bei $r = 5\,\text{ms}$: Bestimmen Sie die absolute Deadline und bewerten Sie richtige Ausgaben bei 12 und 12,2 ms. (2 Punkte)
4. Ein falscher Auftrag liegt schon bei 8 ms vor. Ist der Vertrag erfüllt? (1 Punkt)

Musterlösung: Anfang: festgelegte Erkennung/Freigabe; Ende: richtiger Sperrauftrag am Softwareausgang; $f - r \leq 7\,\text{ms}$. Sperrauftrag Hard, Foto Firm, Statistik Soft. Ausschlaggebend sind Missregel und Restnutzen. $d = 5 + 7 = 12\,\text{ms}$: 12 erfüllt die inklusive Grenze, 12,2 verletzt sie um 0,2 ms. Die falsche Ausgabe bei 8 ms verletzt die funktionale Bedingung trotz erfüllter Zeitbedingung. Punkte entsprechend Teilaufgaben.

Aufgabe 2: Eigene Arbeit und Antwort auseinanderhalten (8 Punkte)

Ein Job wird bei 3 ms freigegeben. Er rechnet von 5 bis 8 und von 10 bis 12 ms, dazwischen wird er unterbrochen. Absolute Deadline: 14 ms. Eine periodische Ausgabe hat die Sollzeitpunkte 0, 8, 16, 24, 32 ms und die Istzeitpunkte 0, 8,3, 16,1, 24,4, 32,2 ms.

1. Zeichnen Sie den Jobverlauf. Berechnen Sie eigene CPU-Zeit, Startverzug, Antwortzeit und Reserve. (4 Punkte)
2. Berechnen Sie die vier Istperioden, Mittelwert und Periodenspanne. (2 Punkte)
3. Berechnen Sie die Spanne der Phasenabweichungen. Weshalb ist das eine andere Größe? (1 Punkt)
4. Beweist das größte Ergebnis einer endlichen Messreihe die WCET? (1 Punkt)

Musterlösung: Rechnen in $[5, 8)$ und $[10, 12)$; eigene CPU-Zeit $3 + 2 = 5$ ms, Startverzug 2 ms, Antwortzeit $12 - 3 = 9$ ms, Reserve $14 - 12 = 2$ ms. Warten/Unterbrechung insgesamt 4 ms. Istperioden: 8,3; 7,8; 8,3; 7,8 ms; Mittelwert 8,05 ms, Spanne 0,5 ms. Phasenabweichungen: 0; 0,3; 0,1; 0,4; 0,2 ms, Spanne 0,4 ms. Phase vergleicht Ist und Soll desselben Ereignisses, Periode benachbarte Istereignisse. Ein Messmaximum deckt nicht automatisch alle zulässigen Pfade/Zustände ab und beweist keine WCET.

Aufgabe 3: Polling unter Last (8 Punkte)

Eine Runde besteht aus Eingangsprüfung mit höchstens 0,2 ms, Reaktion mit höchstens 0,3 ms und Displayarbeit mit höchstens 3,5 ms. Ein Auftrag bleibt bis zur Abfrage gespeichert. Ein Kern, keine Unterbrechungen, höchstens ein relevanter Auftrag; Erkennung am Prüfungsende. Schätzen Sie konservativ mit einer ganzen maximalen Runde bis zur Erkennung plus Reaktion ab. Relative Frist: 6 ms.

1. Bestimmen Sie Runden- und Antwortgrenze. (2 Punkte)
2. Die Displayarbeit wächst auf 9,5 ms. Rechnen Sie erneut und nennen Sie eine konkrete Ereignisphase mit Fristverletzung. (2 Punkte)
3. Ohne Speicherung ist ein Pegel nur von 2 bis 3 ms HIGH; Abfragen erfolgen bei 0, 4 und 8 ms. Wird der Puls erkannt? Was verändert ein Ereignislatch? (2 Punkte)
4. Zerlegen Sie die Displayarbeit in höchstens 0,5-ms-Schritte mit erneuter Eingangsprüfung nach jedem Schritt. Bestimmen Sie die neue Grenze und den benötigten Anwendungszustand. (2 Punkte)

Musterlösung: Runde 4,0 ms, konservativ $R \leq 4,3$ ms: Frist im Modell nachgewiesen. Mit 9,5 ms Display: Runde 10,0 ms, Antwortgrenze 10,3 ms. Eine Anforderung unmittelbar nach der Abfrage vor der langen Displayphase wartet fast 9,5 ms und verletzt 6 ms tatsächlich; eine bloß zu große obere Grenze allein wäre noch kein Missbeweis. Alle drei Pegelabfragen sehen LOW. Ein Latch speichert einen Hinweis, zählt aber nicht beliebig viele Pulse. Mit Teilstücken: Runde 1,0 ms, Antwortgrenze 1,3 ms; Displayfortschritt bzw. nächster Teilabschnitt muss zwischen Runden gespeichert werden.

Aufgabe 4: Timer und Interruptpfad (8 Punkte)

1. Ein Timer hat den bestätigten Kerntakt 80 MHz, $PSC = 7999$ und $ARR = 2499$. Berechnen Sie mit $f_{\mathrm{UPD}} = f_{\mathrm{TIM}}/((PSC + 1)(ARR + 1))$ Updateabstand und vollständige Blinkfrequenz bei einem Toggle pro Update. (3 Punkte)
2. Zwei IRQs haben denselben Präemptionsrang, aber verschiedene Subprioritäten. Kann allein die Subpriorität den laufenden Handler verdrängen? Was bewirkt ein zulässiger dringlicherer Präemptionsrang? (2 Punkte)
3. Ein Flag ist gesetzt. Main liest es, ein zweites Ereignis setzt es erneut, danach löscht Main es. Was geht verloren und was leistet `volatile` hier nicht? (2 Punkte)
4. Frühe Erfassung dauert 1 ms; anschließend vergehen weitere 4 ms bis zur richtigen Reaktion. Prüfen Sie eine End-to-End-Frist von 4 ms. (1 Punkt)

Musterlösung: Updatefrequenz $80 \cdot 10^6/(8000 \cdot 2500) = 4$ Hz, Abstand 250 ms; zwei Toggles pro Zyklus ergeben 2 Hz Blinken. Subpriorität ordnet gleichzeitig anstehende gleichrangige Anforderungen, präemptiert allein aber keinen laufenden gleichrangigen Handler. Ein dringlicherer Präemptionsrang kann das unter den Maskierungsbedingungen. Der zweite Hinweis geht beim Löschen verloren; `volatile` macht die Lese-/Löschfolge nicht unteilbar und aus einem Flag keinen Ereigniszähler. Gesamtantwort 5 ms, Verletzung um 1 ms.

Aufgabe 5: Eine Prüfbehauptung beurteilen

Fünf vollständig erfasste Antworten dauern 4, 5, 4, 7 und 5 ms; erlaubt sind höchstens 6 ms. Berechnen Sie Mittelwert und Maximum. Beurteilen Sie: „Das mittlere Ergebnis ist gut und die LED blinkt, also ist die harte Frist bewiesen.“

Musterlösung: Mittelwert 5 ms, Maximum 7 ms. Ein gültiger Fall verletzt die Frist um 1 ms und widerlegt die Garantie unter diesen Bedingungen. Weitere günstige Läufe heben ihn nicht auf. Blinken belegt nur sichtbare Funktion, keine allgemeine Fristgarantie.

Aufgabe 6: Eine nachleuchtende Tasteranzeige implementieren

Programmieren, etwa 25–35 Minuten. Die CM4-Basis in `code/lab01/` enthält `lab01_button.c/.h` und `button_step.c`. Der eingebaute Taster **B1/PC13** wird bereits mit 20 ms Stabilitätszeit entprellt. **LD1/PB0** ist Ihre Anzeige; **LD2/PE1** wird unabhängig in der Hauptschleife als Statusanzeige bedient. Kein RTOS.

1. Implementieren Sie `Lab01_ButtonStep()`: Jede neue Betätigung schaltet LD1 für 600 Tickwerte ein. Eine weitere Betätigung startet diese Nachleuchtzeit neu; dauerhaftes Halten verlängert sie nicht. Beim Start bereits gedrückter Taster zählt als erste Betätigung.
2. Verwenden Sie `Lab01_ButtonPressed()`, `HAL_GetTick()` und `Lab01_ButtonLed(on)`. Speichern Sie den vorherigen Tasterzustand und den Beginn der letzten Nachleuchtphase. Ein HAL-Tickwert entspricht hier einer Millisekunde.
3. Der Schritt muss sofort zurückkehren: kein `HAL_Delay`, keine Warteschleife. Die vorhandene Hauptschleife ruft ihn wiederholt auf und bedient die Statusanzeige.
4. Prüfen Sie kurzes Drücken, Halten und zwei Betätigungen innerhalb von 600 ms. Prüfen Sie den Zeitvergleich auch auf Papier beim 32-Bit-Überlauf.
5. Erweitern Sie um einen Betätigungszähler im Debugger. Warum ist er kein Zähler aller elektrischen Flanken?

Vorbereitete Basis: Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.

Musterlösung:

Musterlösung zu Aufgabe 6

Die Differenzbildung bleibt beim Überlauf korrekt, wenn unbeobachtete Intervalle kürzer als ein voller 32-Bit-Umlauf sind. Beispiel: Beginn `0xFFFFFF00`, jetzt `0x00000158`: Differenz 600. Die Entprellung erzeugt stabile Zustandswechsel, keinen verlustfreien Flankenzähler.

```
#include "lab01_button.h"

void Lab01_ButtonStep(void)
{
    static int previous, active;
    static uint32_t since;
    const int pressed = Lab01_ButtonPressed();
    const uint32_t now = HAL_GetTick();
    if (pressed && !previous) {
        since = now;
        active = 1;
        ++lab01_presses;
    }
    previous = pressed;
    if (active && (uint32_t)(now - since) >= 600U) {
        active = 0;
    }
    Lab01_ButtonLed(active);
}
```

Halten erzeugt nur die erste steigende Zustandsflanke. Eine zweite Betätigung setzt `since` neu. Die reale LED-Ausschaltung erfolgt erst bei erneuter Ausführung des Schritts; eine Funktionsprüfung beweist keine exakte physische 600-ms-Dauer.